

Timeline 1

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Can you find the flag in this disk image? Wrap what you find in the picoCTF flag format.

Hints

- Create a Sleuthkit MAC timeline!
 - Look at recent timestamps
 - Pay close attention to timestamps near an anti-forensic action
 - Filter only new files by grepping for macb
-

Tools Used

- `7z` – Extract compressed files
- `file` – Determine file type
- `strings` – Extract printable character sequences
- `exiftool` – Read metadata
- `fdisk` – View partition tables
- `mount` – Mount a disk image as a loopback device
- `ls` – List directory contents
- `cat` – Read file contents
- `base64` – Decode Base64-encoded data
- `f1s` – List files in a disk image (Sleuth Kit)
- `mactime` – Create a MAC timeline (Sleuth Kit)
- `icat` – Retrieve a file by inode number (Sleuth Kit)
- `grep` – Filter output

Complete Solution

Step 1: Download and Extract

Download the disk image named `partition4.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x partition4.img.gz
```

This decompresses the gzipped file and creates the raw disk image `partition4.img`.

Step 2: Basic File Inspection

Verify the file type and understand its structure:

```
file partition4.img
```

Expected output:

```
partition4.img: Linux rev 1.0 ext4 filesystem data, UUID=7a00e9da-98f8-4f0f-b257-95edf422d902 (extents) (64bit) (large files) (huge files)
```

The output confirms this is an **ext4 filesystem** (Linux).

Step 3: Search for Visible Text

Disk images often contain strings embedded directly in the data. Extract all readable character sequences and search for the flag:

```
strings partition4.img | grep -i "picoCTF"
```

Output: (no results)

No flag appears yet. We need to dig deeper using forensic timeline analysis.

Step 4: Inspect Metadata

Examine the disk image metadata:

```
exiftool partition4.img
```

Expected output:

```
ExifTool Version Number      : 13.50
File Name                    : partition4.img
Directory                    : .
File Size                    : 490 MB
File Modification Date/Time   : 2025:12:01 23:53:12+02:00
File Access Date/Time        : 2026:05:13 17:06:43+02:00
File Inode Change Date/Time   : 2026:05:13 17:06:39+02:00
File Permissions              : -rw-r--r--
Error                        : First 1024 bytes of file is binary zeros
```

Nothing suspicious here.

Step 5: View Partitions

Use `fdisk` to see the partition structure:

```
fdisk -l partition4.img
```

Expected output:

```
Disk partition4.img: 467 MiB, 489684992 bytes, 956416 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

There are **no explicit partitions**, meaning the filesystem is written directly to the image (a “superfloppy” or raw filesystem image).

Step 6: Mount the Disk Image (Direct Exploration)

Since the image contains an ext4 filesystem directly, we can mount it as a loopback device:

```
sudo mkdir /mnt/timeline1
```

```
sudo mount -o loop partition4.img /mnt/timeline1
```

Note: The `-o loop` option allows Linux to treat a regular file as a block device.

Step 7: Explore the Mounted Image

After mounting, list its contents:

```
ls -alh /mnt/timeline1
```

Output:

```
total 45K
drwxr-xr-x 22 root root 1.0K Dec  1 23:41 .
drwxr-xr-x 11 root root 4.0K May 13 17:09 ..
drwxr-xr-x  2 root root 4.0K Dec  1 23:42 bin
drwxr-xr-x  2 root root 1.0K Nov 11  2025 boot
drwxr-xr-x  2 root root 1.0K Nov 11  2025 dev
drwxr-xr-x 31 root root 3.0K Dec  1 23:50 etc
drwxr-xr-x  3 root root 1.0K Nov 11  2025 home
drwxr-xr-x 10 root root 1.0K Nov 11  2025 lib
drwx-----  2 root root  12K Nov 11  2025 lost+found
drwxr-xr-x  5 root root 1.0K Nov 11  2025 media
drwxr-xr-x  2 root root 1.0K Nov 11  2025 mnt
drwxr-xr-x  2 root root 1.0K Nov 11  2025 opt
drwxr-xr-x  2 root root 1.0K Nov 11  2025 proc
drwx-----  2 root root 1.0K Dec  1 23:50 root
drwxr-xr-x  2 root root 1.0K Nov 11  2025 run
drwxr-xr-x  2 root root 5.0K Nov 11  2025 sbin
drwxr-xr-x  2 root root 1.0K Nov 11  2025 srv
drwxr-xr-x  2 root root 1.0K Dec  1 23:41 swap
drwxr-xr-x  2 root root 1.0K Nov 11  2025 sys
drwxrwxrwt  2 root root 1.0K Nov 11  2025 tmp
drwxr-xr-x  8 root root 1.0K Nov 11  2025 usr
drwxr-xr-x 11 root root 1.0K Dec  1 23:41 var
```

After exploring, something interesting appears in the `etc` directory.

Step 8: Examine the etc Directory

```
ls -alh /mnt/timeline1/etc
```

Output (abbreviated):

```
total 107K
drwxr-xr-x 31 root root 3.0K Dec  1 23:50 .
drwxr-xr-x 22 root root 1.0K Dec  1 23:41 ..
drwxr-xr-x  3 root root 1.0K Nov 11  2025 acpi
-rw-r--r--  1 root root    7 Oct  8  2025 alpine-release
drwxr-xr-x  4 root root 1.0K Dec  1 23:42 apk
...
-rw-r--r--  1 root root   49 Dec  1 23:50 chat
...
```

Suspicious file detected: `chat` (49 bytes) – an unusual file to find in `/etc` .

Step 9: Read the Suspicious File

```
cat /mnt/timeline1/etc/chat
```

Output:

```
NTczNDE3aDEzcl83aDRuXzdoM18xNDU3XzU4NTI3YmIyMjIK
```

This appears to be **Base64-encoded** data.

Step 10: Decode the Base64

```
echo 'NTczNDE3aDEzcl83aDRuXzdoM18xNDU3XzU4NTI3YmIyMjIK' | base64 -d
```

Output:

<REDACTED>

This reveals the flag!

Alternative Method: Sleuth Kit Timeline Analysis

The hints suggest creating a **MAC timeline** using The Sleuth Kit.

Step A: Extract File List with fls

```
fls -m / -r partition4.img > output.txt
```

Option	Description
-m /	Output in body file format for mactime
-r	Recursively list all contents

Step B: Create Chronological Timeline with mactime

```
mactime -b output.txt > timeline.txt
```

This creates `timeline.txt` sorted by timestamp (oldest to newest).

Step C: Understand MACB Timestamps

`mactime` output includes a column with letters **m a c b** representing the four timestamps:

Letter	Meaning
m	Modify – data modification time
a	Access – file access time
c	Change – metadata change time (permissions, ownership)

Letter	Meaning
b	Birth – file creation time

💡 **Anti-forensic indicator:** When a tool intentionally modifies timestamps (timestamp abuse), all four flags (`macb`) may appear together, overwritten with the same time.

Step D: Filter Recent `macb` Entries

```
grep "macb" timeline.txt | tail -n 30
```

Output (excerpt):

```

                                45472 ...b r/rrw-r--r-- 0          0          2339
/lib/modules/6.6.116-0-lts/kernel/drivers/net/ethernet/cadence/macb.ko.gz
                                0 macb r/rrw----- 0          0          19
/lib/apk/db/lock
                                45472 ..c. r/rrw-r--r-- 0          0          2339
/lib/modules/6.6.116-0-lts/kernel/drivers/net/ethernet/cadence/macb.ko.gz
                                45472 .a.. r/rrw-r--r-- 0          0          2339
/lib/modules/6.6.116-0-lts/kernel/drivers/net/ethernet/cadence/macb.ko.gz
                                49 macb r/rrw-r--r-- 0          0          32716
/etc/chat
                                8 macb r/rrw-r--r-- 0          0          33020
/lib/rc/cache/softlevel
```

Key finding: The file `/etc/chat` (inode **32716**) has all `macb` flags present – a strong indicator of anti-forensic manipulation!

Step E: Extract the File by Inode

Use `icat` to retrieve the file directly by its inode number:

```
icat partition4.img 32716
```

Output:

```
NTczNDE3aDEzc183aDRuXzdoM18xNDU3XzU4NTI3YmIyMjIK
```

Same Base64 string! Decode it to get the flag.

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, decoding the Base64 string will reveal the complete flag.

Key Concepts

- **Raw Disk Image:** A sector-by-sector copy of a storage device that can contain a filesystem directly.
- **ext4 Filesystem:** The native Linux filesystem used in this image.
- **Loopback Mount:** Mounting a file as a block device using `mount -o loop .`
- **MAC Timestamps:** Modify, Access, Change, Birth – four timestamps tracked by Linux filesystems.
- **Anti-Forensic Techniques:** Methods to hide or alter evidence, such as timestamp manipulation.
- **macb Indicator:** When all four timestamps are identical or appear together, it may indicate tampering.
- **Sleuth Kit (TSK):** A collection of command-line tools for forensic analysis of disk images.
- **fls:** Lists files and directories in a disk image.
- **mactime:** Creates a chronological timeline from body files.
- **icat:** Retrieves a file by its inode number without mounting the filesystem.